Name	Anushka Pradip Chaudhari
UID no.	2024300027
Experiment No.	3

TITLE:	To implement network socket programming for establishing communication between client and server over a network.
PROGRAM:	**PROGRAM 1: To implement a client-server program where the client sends a message to the server and the server receives and displays it.** **Code:** **1. Server.c**

```
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>
#define PORT 8080
int main(int argc, char const* argv[])
{
    int server_fd, new_socket;
    ssize_t valread;
    struct sockaddr_in address;
    int opt = 1;
    socklen_t addrlen = sizeof(address);
    char buffer[1024] = { 0 };
    char* hello = "Hello from server";

    // Creating socket file descriptor
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }
```

```
// Forcefully attaching socket to the port 8080
if (setsockopt(server_fd, SOL_SOCKET,
               SO_REUSEADDR | SO_REUSEPORT, &opt,
               sizeof(opt))) {
    perror("setsockopt");
    exit(EXIT_FAILURE);
}
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

// Forcefully attaching socket to the port 8080
if (bind(server_fd, (struct sockaddr*)&address,
         sizeof(address))
    < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}
if (listen(server_fd, 3) < 0) {
    perror("listen");
    exit(EXIT_FAILURE);
}
if ((new_socket
     = accept(server_fd, (struct sockaddr*)&address,
              &addrlen))
    < 0) {
    perror("accept");
    exit(EXIT_FAILURE);
}

// subtract 1 for the null
// terminator at the end
valread = read(new_socket, buffer,
               1024 - 1);
printf("%s\n", buffer);
send(new_socket, hello, strlen(hello), 0);
printf("Hello message sent\n");

// closing the connected socket
```

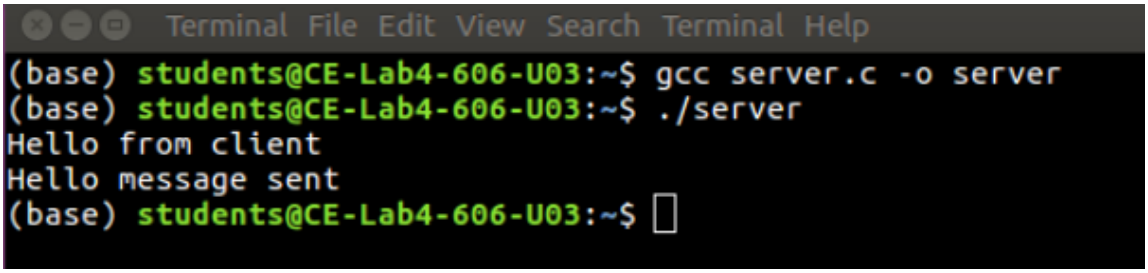
```
    close(new_socket);

    // closing the listening socket
    close(server_fd);
    return 0;
}
```

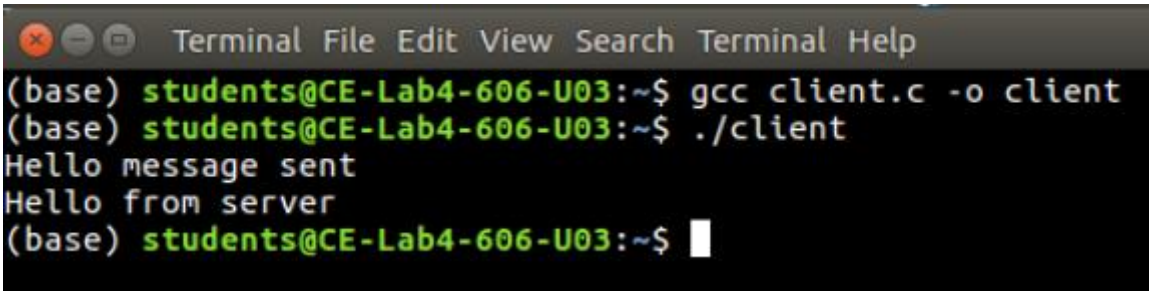
2. Client.c

```
#include <arpa/inet.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>
#define PORT 8080

int main(int argc, char const* argv[])
{
    int status, valread, client_fd;
    struct sockaddr_in serv_addr;
    char* hello = "Hello from client";
    char buffer[1024] = { 0 };
    if ((client_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 and IPv6 addresses from text to binary
    // form
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr)
        <= 0) {
        printf(
            "\nInvalid address/ Address not supported \n");
        return -1;
    }

    if ((status
```

```
        = connect(client_fd, (struct sockaddr*)&serv_addr,
                  sizeof(serv_addr)))
      < 0) {
      printf("\nConnection Failed \n");
      return -1;
   }

   // subtract 1 for the null
   // terminator at the end
   send(client_fd, hello, strlen(hello), 0);
   printf("Hello message sent\n");
   valread = read(client_fd, buffer,
                  1024 - 1);
   printf("%s\n", buffer);

   // closing the connected socket
   close(client_fd);
   return 0;
}
```

OUTPUT:





PROGRAM 2: To develop a client-server application where the client sends two numbers to the server, the server performs addition, and returns the result to the client.

Code:

1. Server.c

```
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define PORT 8080

int main()
{
   int server_fd, new_socket;
   struct sockaddr_in address;
   int opt = 1;
   socklen_t addrlen = sizeof(address);

   int numbers[2];   // To store two numbers
   int result;

   // Create socket
   if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
      perror("socket failed");
      exit(EXIT_FAILURE);
   }

   // Set socket options
   if (setsockopt(server_fd, SOL_SOCKET,
                  SO_REUSEADDR | SO_REUSEPORT,
                  &opt, sizeof(opt))) {
      perror("setsockopt");
      exit(EXIT_FAILURE);
   }
```

```
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

// Bind
if (bind(server_fd, (struct sockaddr*)&address,
        sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}

// Listen
if (listen(server_fd, 3) < 0) {
    perror("listen");
    exit(EXIT_FAILURE);
}

printf("Waiting for connection...\n");

// Accept connection
if ((new_socket = accept(server_fd,
                    (struct sockaddr*)&address,
                    &addrlen)) < 0) {
    perror("accept");
    exit(EXIT_FAILURE);
}

// Read two integers from client
read(new_socket, numbers, sizeof(numbers));

printf("Received numbers: %d and %d\n",
       numbers[0], numbers[1]);

// Add them
result = numbers[0] + numbers[1];

// Send result back
send(new_socket, &result, sizeof(result), 0);

printf("Result sent: %d\n", result);
```

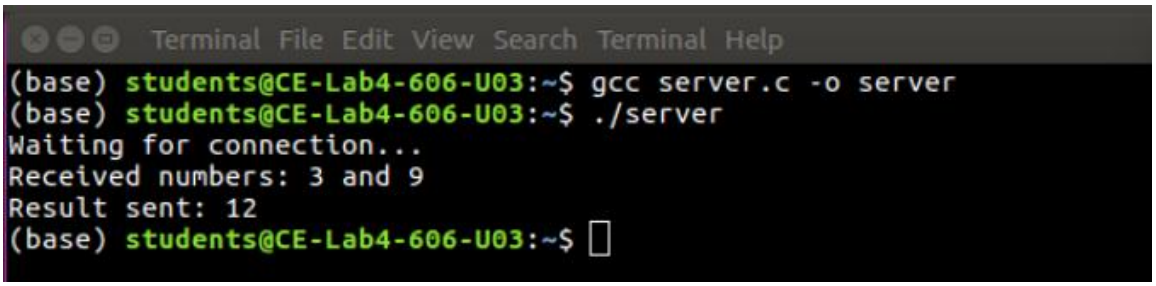
```
    close(new_socket);
    close(server_fd);

    return 0;
}
```

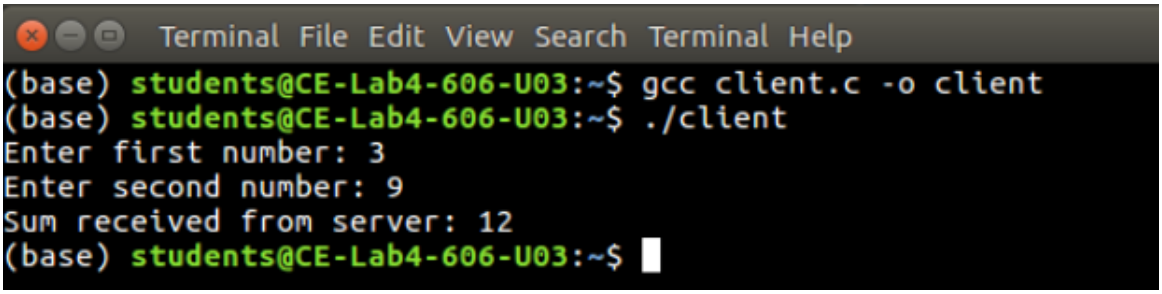
2. Client.c

```
#include <arpa/inet.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define PORT 8080

int main()
{
    int client_fd;
    struct sockaddr_in serv_addr;

    int numbers[2];
    int result;

    // Create socket
    if ((client_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("Socket creation error\n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    if (inet_pton(AF_INET, "127.0.0.1",
                  &serv_addr.sin_addr) <= 0) {
        printf("Invalid address\n");
        return -1;
    }

    // Connect
```

```
    if (connect(client_fd,
                (struct sockaddr*)&serv_addr,
                sizeof(serv_addr)) < 0) {
        printf("Connection Failed\n");
        return -1;
    }
    printf("Enter first number: ");
    scanf("%d", &numbers[0]);

    printf("Enter second number: ");
    scanf("%d", &numbers[1]);

    send(client_fd, numbers, sizeof(numbers), 0);

    // Receive result
    read(client_fd, &result, sizeof(result));

    printf("Sum received from server: %d\n", result);

    close(client_fd);

    return 0;
}
```

OUTPUT:





PROGRAM 3: To design a multi-client communication system where one client sends a message to another client through the server.

Code:

1. Server.c

```
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main()
{
   int server_fd, client1, client2;
   struct sockaddr_in address;
   int opt = 1;
   socklen_t addrlen = sizeof(address);
   char buffer[BUFFER_SIZE];

   // Create socket
   server_fd = socket(AF_INET, SOCK_STREAM, 0);

   setsockopt(server_fd, SOL_SOCKET,
            SO_REUSEADDR | SO_REUSEPORT,
            &opt, sizeof(opt));

   address.sin_family = AF_INET;
   address.sin_addr.s_addr = INADDR_ANY;
   address.sin_port = htons(PORT);

   bind(server_fd, (struct sockaddr*)&address,
       sizeof(address));

   listen(server_fd, 3);
```

```
    printf("Waiting for Client 1...\n");
    client1 = accept(server_fd,
                     (struct sockaddr*)&address,
                     &addrlen);
    printf("Client 1 connected\n");

    printf("Waiting for Client 2...\n");
    client2 = accept(server_fd,
                     (struct sockaddr*)&address,
                     &addrlen);
    printf("Client 2 connected\n");

    // Receive from Client1
    read(client1, buffer, BUFFER_SIZE);
    printf("Received from Client1: %s\n", buffer);

    // Send to Client2
    send(client2, buffer, strlen(buffer), 0);
    printf("Message forwarded to Client2\n");

    close(client1);
    close(client2);
    close(server_fd);

    return 0;
}
```

2. Client1.c

```
#include <arpa/inet.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define PORT 8080
#define BUFFER_SIZE 1024
int main()
{
    int sock;
```

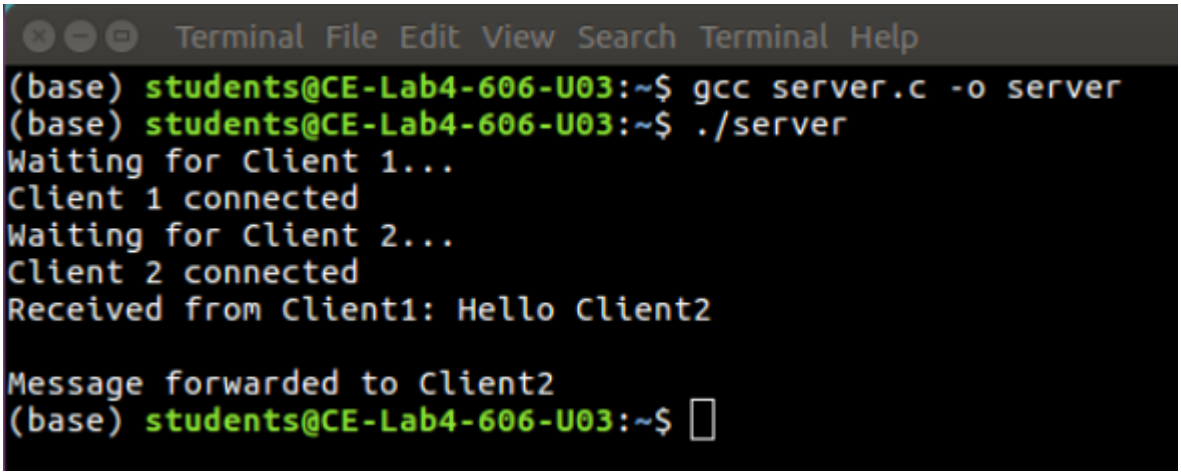
```
    struct sockaddr_in serv_addr;
    char message[BUFFER_SIZE];
    sock = socket(AF_INET, SOCK_STREAM, 0);
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1",
              &serv_addr.sin_addr);
    connect(sock, (struct sockaddr*)&serv_addr,
            sizeof(serv_addr));
    printf("Enter message to send to Client2: ");
    fgets(message, BUFFER_SIZE, stdin);
    send(sock, message, strlen(message), 0);
    printf("Message sent to server\n");
    close(sock);
    return 0;
}
```

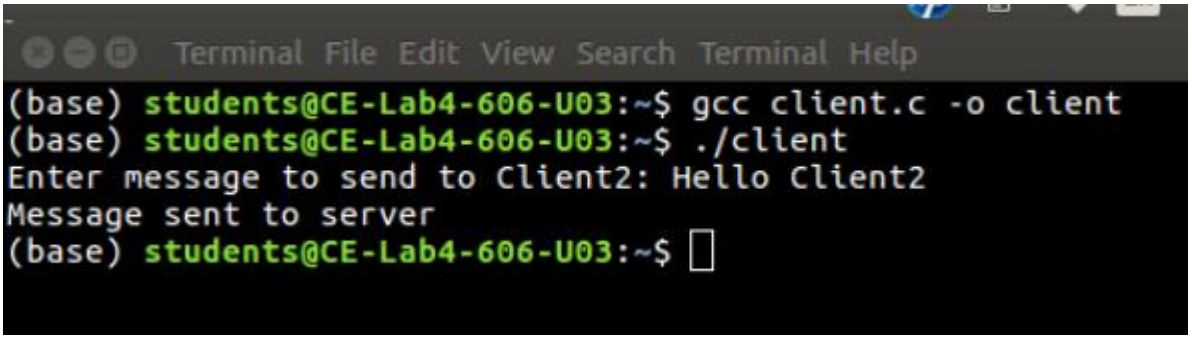
3. Client2.c

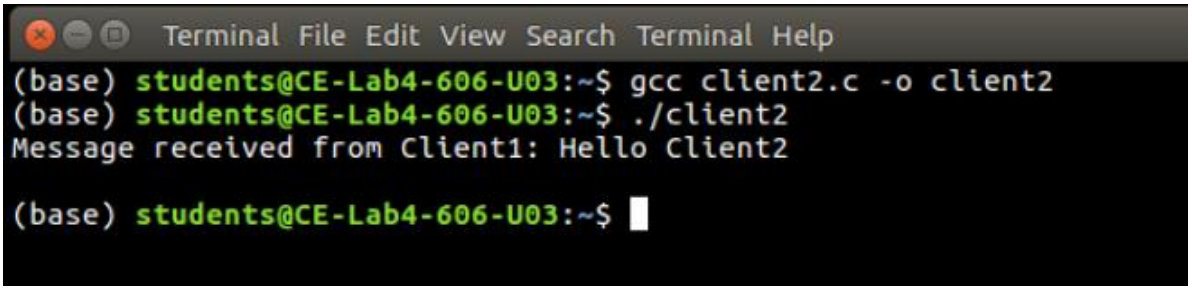
```
#include <arpa/inet.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define PORT 8080
#define BUFFER_SIZE 1024
int main()
{
    int sock;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    sock = socket(AF_INET, SOCK_STREAM, 0);
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1",
              &serv_addr.sin_addr)
    connect(sock, (struct sockaddr*)&serv_addr,
            sizeof(serv_addr));
    read(sock, buffer, BUFFER_SIZE);
```

```
    printf("Message received from Client1: %s\n", buffer);
    close(sock);
    return 0;
}
```

OUTPUT:







CONCLUSION: The experiment successfully demonstrates client-server communication using network socket programming. It validates message transfer, remote computation, and communication between multiple clients through a server, forming the basic foundation of network-based applications.